# CS23532 - COMPUTER NETWORKS LABORATORY

EXPERIMENT 6

## AIM:

To implement the Hamming code concept for Error Correction at the Data Link Layer by writing a program to detect and correct errors, and verifying its functionality through a test run with an input data stream.

## DESCRIPTION:

- **Hamming Code:** A technique developed by R.W. Hamming that utilizes a set of error-correction codes to identify and fix errors that may occur when data is transmitted from a sender to a receiver.
- **Error Correction at Data Link Layer:** It involves adding redundant bits (parity bits) to the original data before transmission. These bits allow the receiver to mathematically detect if a bit was flipped during transmission and correct single-bit errors without needing the sender to retransmit the data.

## STEP-BY-STEP PROCEDURE:

### Task 1: Implement and Execute the Sender Program

1. Write the sender script to accept a text string input of any length.
2. Convert the input string into its corresponding binary format.
3. Calculate the required number of redundant parity bits based on the data length.
4. Insert the redundant bits at the correct power-of-2 positions and calculate their values (even/odd parity).
5. Save the resulting encoded binary data stream into a text file named "channel".

### Task 2: Simulate a Transmission Error (Optional but recommended for testing)

1. Open the "channel" file manually.

2. Alter exactly one bit (change a 0 to a 1, or a 1 to a 0) to simulate noise in the transmission medium.

Save and close the file.

3. Task 3: Execute Receiver Program to Detect & Correct Error
4. Run the receiver script to process the bitstream from "channel", verify parity, fix any detected single-bit error, and output the reconstructed string.

# PROGRAM CODE AND LOGIC:

## Sender Script (sender.py)

Python

```
import math

def text_to_binary(text):
    return ''.join(format(ord(c), '08b') for c in text)

def encode_hamming(data_bits):
    m = len(data_bits)
    r = 0
    while (2**r) < (m + r + 1):
        r += 1

    n = m + r
    code = ['0'] * (n + 1)
    j = 0
    for i in range(1, n + 1):
        if (i & (i - 1)) == 0:
            code[i] = '0'
        else:
            code[i] = data_bits[j]
            j += 1

    for i in range(r):
        p_pos = 2**i
        parity = 0
        for k in range(1, n + 1):
```

```
            if k & p_pos:
                parity ^= int(code[k])
        code[p_pos] = str(parity)

    return ''.join(code[1:])

if __name__ == "__main__":
    user_input = input("Enter text to send: ")
    binary_data = text_to_binary(user_input)
    encoded_data = encode_hamming(binary_data)
    with open("channel", "w") as f:
        f.write(encoded_data)
    print("Encoded data written to 'channel'.")
```

## Receiver Script (receiver.py)

Python

```
import math

def decode_hamming(code_str):
    n = len(code_str)
    r = 0
    while (2**r) <= n:
        r += 1

    code = ['0'] + list(code_str)
    error_pos = 0

    for i in range(r):
        p_pos = 2**i
        parity = 0
        for k in range(1, n + 1):
            if k & p_pos:
                parity ^= int(code[k])
        if parity != 0:
```

```
            error_pos += p_pos

    if error_pos != 0:
        print(f"Error detected at bit position: {error_pos}")
        code[error_pos] = '0' if code[error_pos] == '1' else '1'
        print("Error corrected successfully.")
    else:
        print("No errors detected.")

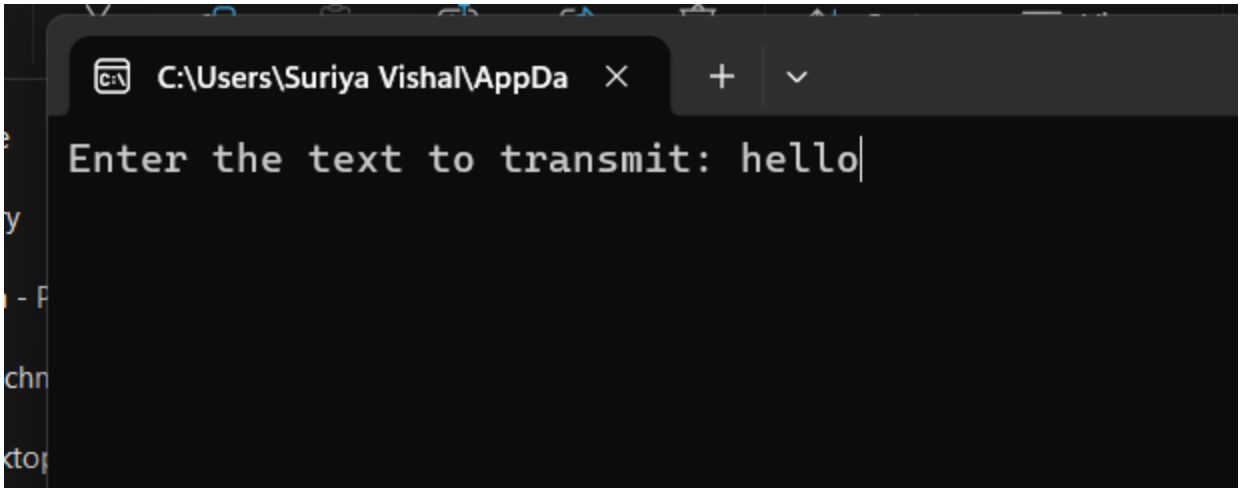
    data_bits = []
    for i in range(1, n + 1):
        if (i & (i - 1)) != 0:
            data_bits.append(code[i])

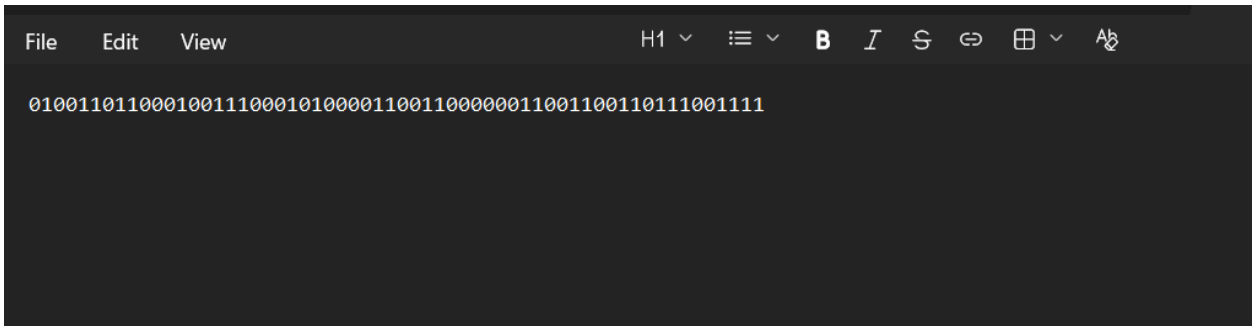
    return ''.join(data_bits)

def binary_to_text(binary_str):
    chars = []
    for i in range(0, len(binary_str), 8):
        byte = binary_str[i:i+8]
        if len(byte) == 8:
            chars.append(chr(int(byte, 2)))
    return ''.join(chars)

if __name__ == "__main__":
    try:
        with open("channel", "r") as f:
            received_code = f.read().strip()
        recovered_binary = decode_hamming(received_code)
        recovered_text = binary_to_text(recovered_binary)
        print(f"Decoded Message: {recovered_text}")
    except FileNotFoundError:
        print("Error: 'channel' file not found.")
```

Verification:







## Result:

To implement the Hamming code concept for Error Correction at the Data Link Layer by writing a program to detect and correct errors, and verifying its functionality through a test run has been completed successfully.